

Operation — Consent Trap

OAuth consent abuse against cloud identity · Ref RIU-INT-2026-1008-CT · OSINT + defensive analysis only · TLP:AMBER

HIGH confidence: malicious consent	NONE attribution	2 users same app ID in 39 min	+21 min consent → mailbox read	340 + 23 mail items + files	20/20 tests pass
--	----------------------------	---	--	---------------------------------------	----------------------------

CTI Lead's question — "legitimate SaaS, or identity-based intrusion through OAuth consent?" Identity-based intrusion through OAuth consent — confidence HIGH, attribution NONE. A first-seen application from an unverified publisher received user consent for mail + files + `offline_access`; 7 minutes later it called the API from a German hosting IP, at +21 min it pulled 340 mail items, at +26 min it read 23 files on two sites the user never opens, and 39 minutes after the first grant a second employee consented to the same application ID from the same lure domain. No password was stolen and MFA was never bypassed — the user approved the access on the genuine Microsoft consent screen.

Mission 01 — Campaign Intel Card

Field	Intelligence
Threat activity	OAuth consent phishing ("illicit consent grant"): the victim is led to authorise an attacker-controlled application instead of typing a password
Target	Microsoft 365 tenants that leave default user consent on; inside them — finance, HR, executives, shared-mailbox owners
Initial access	Document-sharing lure (email/Teams) → link on a look-alike domain → redirect to the real identity-provider <code>/authorize</code> page carrying the attacker's <code>client_id</code>
Access mechanism	Delegated permissions + refresh token. The app acts <i>as the user</i> through the API from its own infrastructure — no interactive sign-in afterwards
Threat actors	Unattributed. Technique is used across the spectrum — financially motivated BEC crews, phishing-as-a-service operators, access brokers and state-aligned groups (Microsoft documented OAuth-app abuse in the January 2024 Midnight Blizzard intrusion). No evidence links this campaign to any of them
Capability	Multi-tenant app registration, convincing branding, automated mail/file collection over Graph, long-lived access via <code>offline_access</code>
Infrastructure	<code>secure-docverify.example.net</code> (registered 6 days before use, privacy-protected, one certificate), hosting IP 203.0.113.45 (DE), two co-hosted look-alike domains
Motivation	Assessed as data theft / BEC preparation (finance mail, treasury and board documents read first) — Medium confidence
Confidence	High that this is malicious consent · Low on who is behind it

Research notes behind the card - Password theft vs OAuth abuse: a stolen password must still pass MFA and conditional access at sign-in, and a reset ends it. A consent grant produces tokens *issued by the tenant itself*; the user's password, MFA and even a later password reset do not touch the grant. - **Why MFA alone does not stop it:** MFA protects the *sign-in*. Here the user signs in legitimately (or already has a session) and then *authorises a third party*. The app's later API calls are non-interactive, so there is nothing for MFA to challenge. - **How apps are presented:** trusted-sounding names ("Secure...", "...Verification", "...Docs"), a familiar document-sharing pretext, and the credibility of the real Microsoft consent page. Public reporting (Proofpoint, Jan 2023) showed attackers even obtaining "verified publisher" status to look legitimate. - **Permissions sought:** `Mail.Read*`, `Files.Read.All` / `Sites.Read.All`, `MailboxSettings.ReadWrite` (inbox rules), `Mail.Send`, `User.ReadBasic.All` (internal recon) and almost always `offline_access`. - **Compromised legitimate apps:** a genuine vendor's app that already holds tenant-wide grants becomes an attacker's token factory if the vendor's secret or signing key leaks — the customer sees only "the approved app" behaving differently. - **Most exposed organisations:** default user-consent setting, many third-party SaaS integrations, no app-governance review, short audit retention, heavy use of shared mailboxes/SharePoint for sensitive finance and HR data. - **Persistence:** refresh tokens, plus any inbox rule or extra permission the app adds. Access lasts until the grant is revoked and the service principal disabled. - **Evidence of malicious consent:** first-seen app ID + unverified publisher + data scopes + `offline_access` + user (not admin) consent, followed by API activity from non-user infrastructure and data access outside the user's baseline. - **Why cloud identity telemetry matters to CTI:** the only indicators that exist are in the identity plane — app ID, publisher, reply URL, scopes, consent time, calling IP. There is no malware hash and nothing on the endpoint.

Mission 02 — Threat actor & TTP mapping (MITRE ATT&CK)

Technique	ID	Observed evidence (lab)	How it applies	Confidence
Phishing: Spearphishing Link	T1566.002	proxy: click on <code>secure-docverify.example.net/share/...</code> at 08:41:30	document-sharing lure delivers the consent link	High
User Execution: Malicious Link	T1204.001	user clicked, then approved	the grant needs the user's own action	High
Steal Application Access Token	T1528	<code>Consent to application</code> 08:42:11, scopes incl. <code>offline_access</code>	consent = the user hands the app a token	High

Technique	ID	Observed evidence (lab)	How it applies	Confidence
Use Alternate Auth Material: Application Access Token	T1550.001	Graph calls from 203.0.113.45 with no interactive sign-in	token replayed from attacker infrastructure	High
Valid Accounts: Cloud Accounts	T1078.004	activity is recorded under <code>finance.user</code> / <code>hr.user</code>	actions inherit the user's identity and rights	High
Cloud Service Discovery / Dashboard	T1526 / T1538	first calls <code>/me</code> , <code>/me/mailFolders</code>	orientation before collection (API-level; no portal use seen)	Medium
Email Collection: Remote Email Collection	T1114.002	<code>MailItemsAccessed</code> ×340 from DE	mailbox harvested over the API	High
Data from Information Repositories: SharePoint	T1213.002	23 files on Finance-Treasury, Board-Packs	file collection outside baseline	High
Account Manipulation / Additional Cloud Roles	T1098 / T1098.003	not observed — no role or permission added after consent	hunt item for the 7-day plan	Low
Exfiltration Over Web Service	T1567	downloads to app infrastructure	inferred from <code>FileDownloaded</code> — volume unknown	Medium

Senior TTP question — is the application the attack, or the mechanism? The mechanism. The attack *is persistent, delegated access to the victim's cloud identity*; the application is the vehicle that carries the token, exactly as a phishing page is the vehicle for a password. That distinction drives response: deleting or blocking the app name is not enough — an actor re-registers under a new name in minutes. What must be removed is the **grant and its refresh tokens**, and what must be fixed is the **consent policy** that allowed any user to issue such a grant.

Mission 03 — OSINT & application exposure intelligence (OSINT only; lab values simulated)

OSINT query / method	Data source	Intelligence obtained (simulated)	Valid indicator	False positive
Domain registration date + registrar for the reply-URL domain	WHOIS / RDAP	registered 02 Oct 2026, privacy-protected — 6 days before first consent	domain younger than ~30 days used as an OAuth redirect	new domain of a genuine start-up vendor with a real company record
Certificates issued for the domain and siblings	Certificate Transparency (crt.sh)	first certificate 03 Oct; names <code>secure-docsign...</code> , <code>hr-docverify...</code> on similar pattern	cluster of look-alike names issued within days	wildcard certs of a large CDN/SaaS host
Passive DNS / co-hosting on the IP	VirusTotal relations, URLScan	two more look-alike domains on 203.0.113.45	several "doc-verify/sign" domains on one small host	shared hosting with thousands of unrelated sites
Page capture of the lure without visiting it	URLScan	title "Sign in to view", immediate redirect to the IdP with a fixed <code>client_id</code>	redirect chain embeds the same application ID seen in the tenant	marketing redirectors of real SaaS (known <code>client_id</code>)
Reputation of domain / IP	VirusTotal	0 / 90 — clean because new	<i>nothing</i> — a clean score is not evidence of safety	treating "0 detections" as legitimate
Publisher verification + app metadata	Entra consent record, Microsoft docs	publisher unverified; no homepage, privacy policy or support contact	unverified publisher asking for mail/file scopes	internal line-of-business app (your own tenant ID as owner)
Does the vendor exist?	company registry, app marketplace, LinkedIn	no company, no marketplace listing	brand that exists only on the lure domain	small real vendor — confirm with procurement
Has this app/domain been reported?	vendor blogs, CISA advisories, MITRE	no prior report (new infrastructure)	match on application ID or redirect domain in a public report	name collision — generic names are reused by benign apps

Answers to the research questions: registered 02 Oct 2026 · resembles a generic "document verification" service rather than one specific vendor (trust-bait, not typosquat) · publisher not trustworthy (unverified, no footprint) · same infrastructure hosts two related domains · no historical reports · branding does not match the claimed purpose — a "verification" tool has no need to read a whole mailbox and every accessible file. **Rule:** an indicator is valid only when it ties back to the tenant's own evidence (application ID, redirect URI, calling IP); a display name alone never is.

Mission 04 — Cloud Application Compromise Hunt Table

Time (IST)	User	Application	Event	Risk	Why?
08:15:00	sales.user	Example PDF Reader	OAuth consent	● NORMAL	verified publisher, <code>User.Read</code> only — a sign-in grant (false-positive control)
08:42:11	finance.user	Secure Document Verification	OAuth consent — Mail + Files	● SUSPICIOUS	first-seen app, unverified publisher, data scopes + <code>offline_access</code> , user consent. Alone it could still be a new SaaS
08:49:32	finance.user	same	API activity ×12	● SUSPICIOUS	7 min after consent, from 203.0.113.45 (DE) — user baseline IN, no interactive sign-in
09:03:18	finance.user	same	Mailbox access ×340	● HIGH PRIORITY	bulk mail read via app token from Germany, 21 min after consent

Time (IST)	User	Application	Event	Risk	Why?
09:07:42	finance.user	same	File access ×23	● HIGH PRIORITY	Finance-Treasury + Board-Packs — sites outside her 30-day baseline
09:21:11	hr.user	same	OAuth consent	● HIGH PRIORITY	second user, same application ID, same lure domain → campaign, not coincidence
09:31:52	finance.user + hr.user	same	API activity ×86	● HIGH PRIORITY	one app, one IP, multiple users simultaneously

The same tool scores 23 activity rows of the two approved applications as NORMAL, and the admin-consented new SaaS (TeamBoard Planner, ticket PRC-1240) is not flagged.

Mission 05 — IoC & detection intelligence

Indicator type	Value (synthetic)
Application name / ID	Secure Document Verification · 5d0c0000-0000-4000-8000-00000000c7a9 — hunt on the ID, names change
Publisher	SecureDocs Verification Ltd — unverified
Redirect URI / lure domain	https://secure-docverify.example.net/callback · /share/*
Related domains	secure-docsign.example.net, hr-docverify.example.org
Calling infrastructure	203.0.113.45 (DE hosting)
High-risk permission set	Mail.ReadBasic + Files.Read.All + offline_access by user consent
Behavioural	first-seen app activity · ≥2 users consenting to one app in 1 h · API calls with no interactive sign-in · mail volume 8× baseline · files on never-visited sites

Detection hypothesis — "new OAuth application consent followed by unusual cloud activity": *if* a non-admin user consents to an application that is not on the approved list and carries data scopes, *and* within 60 minutes that application ID acts for the same user from a country outside the user's baseline *or* touches mail/files outside the user's baseline, *then* the consent is malicious until proven otherwise.

User	App ID	Publisher	Permissions	Consent (IST)	Source IP / country	API	Mail	Files	Δ first API	Δ first data	Fired
finance.user	...c7a9	unverified	Mail, Files, offline	08:42:11	203.0.113.45 / DE	67	340	23	7.3 min	21.1 min	✓
hr.user	...c7a9	unverified	Mail, Files, offline	09:21:11	203.0.113.45 / DE	31	0	0	10.7 min	—	✓

Shipped as Sigma (oauth_consent_abuse.sigma.yml — 3 rules + 2 correlations) and KQL reference hunts (consent_hunt.kql).

False-positive analysis: a new company-approved SaaS, an HR/Finance tool roll-out, marketplace add-ins, security tools, automation platforms and approved integrations all create first-seen consents and foreign API traffic. They are separated by: admin consent with a ticket, a verified publisher with a real vendor footprint, scopes that match the product's purpose, stable vendor IP ranges, and data access that stays inside what users already touch.

What would raise confidence further: the lure email itself with the same client_id; the same application ID in other tenants or public reporting; an inbox rule or Mail.Send use; token use continuing after the user's password reset; data access with no relation to a "document verification" function; the vendor failing to exist when procurement checks.

Mission 06 — Diamond Model

Vertex	Assessment	Confidence
 Adversary	Unknown. Consistent with a financially motivated phishing operator or an access broker preparing BEC (finance mail and treasury files first). A cloud-focused intrusion group cannot be excluded	Low
 Capability	Lure delivery · multi-tenant OAuth app · delegated token + refresh token · automated Graph collection of mail and files · reuse across users within 40 minutes	High (directly observed)
 Infrastructure	Look-alike domain registered days earlier · single hosting IP for redirect and API calls · sibling domains staged for other themes (HR, e-sign) · the victim's own identity provider used as the consent page	Medium (domain/IP observed; siblings via simulated OSINT)
 Victim	4,500-employee M365 tenant with default user consent; finance and HR users with access to treasury, board and employee data	High

Adversary → Capability → Infrastructure → Victim: an unknown operator uses consent phishing, hosted on freshly registered look-alike infrastructure, to obtain delegated tokens for finance and HR identities.

Intelligence gaps: the original lure message and sender; whether more users clicked without consenting; total volume actually downloaded; whether the same application ID appears in other organisations; what the app did before audit visibility (if retention is short). **Evidence required before attribution:** infrastructure or application-ID overlap with a *named, publicly documented* cluster, corroborated by a second independent source and by matching tradecraft (lure themes, scope sets, timing). Shared hosting or a similar app name is not attribution.

Mission 07 — Attack chain analysis

Stage	Attacker objective	MITRE	Expected evidence	Possible detection	SOC response
Phishing / social engineering	get the link in front of a finance user	T1566.002	mail gateway logs, URL rewrite logs	new-domain URL in mail; look-alike domain feed	block domain, pull the message from all mailboxes
User interaction	click through to the consent page	T1204.001	proxy: lure URL → IdP /authorize?client_id=...	redirect to IdP carrying an unknown client_id	identify every user who clicked
OAuth consent	obtain token + refresh token	T1528	Entra audit Consent to application, scopes, consent type	user consent + unverified publisher + data scopes	revoke grant, disable service principal
Cloud access	act as the user	T1550.001, T1078.004	non-interactive / service-principal sign-ins, Graph activity	app ID calling from non-baseline country	revoke refresh tokens and sessions
Discovery	learn mailbox and file layout	T1526	/me, folder and drive listings	first-seen app enumerating folders	scope what was listed
Email / file collection	take the data	T1114.002, T1213.002	MailItemsAccessed, FileAccessed/Downloaded	volume and site outside user baseline	build the exposure list for data owners
Persistence	keep access	T1098 (hunt)	refresh-token use over days, inbox rules, added permissions	token use after password reset; new rules	remove rules, confirm grant is gone
Data theft / impact	BEC, fraud, leak	T1567, T1657	look-alike invoice mails, payment-detail changes	finance process controls, vendor call-backs	warn treasury, start breach assessment

Where is the strongest opportunity to stop it? - Before consent — the cheapest but least reliable point. Mail filtering and awareness reduce clicks, yet new domains score clean and the consent page is genuine. Stops some, never all. - **At consent — the strongest control point.** One tenant setting (users cannot consent to unverified publishers or to anything beyond low-risk scopes; everything else goes to an admin-consent request) makes the attack fail for every user, every lure and every future app name, with no detection needed. It is the only stage fully under the defender's control. - **After consent** — the best *detection* point (rich telemetry: app ID, IP, volumes) but the clock is already running: here mail was read 21 minutes after the grant. Response at this stage limits damage; it does not prevent it. So: **prevent at consent, detect after consent, educate before consent** — in that order of return on effort.

Mission 08 — Response intelligence (assume the application is malicious)

Next 24 hours	Next 7 days	Next 30 days
☒ Affected users: <code>finance.user</code> , <code>hr.user</code> (plus anyone who clicked the lure domain)	☐ Historical consent hunt — every user consent in the audit retention period	☐ Restrict user consent: verified publishers + low-risk scopes only; admin-consent workflow for the rest
☒ Application ID <code>...c7a9</code> — search tenant-wide by ID	☐ Search the app ID, publisher and redirect domain across all logs	☐ Review every third-party app: owner, purpose, scopes, last use; remove unused grants
☒ Review requested permissions; revoke the delegated grants	☐ Hunt related domains (<code>secure-docsign...</code> , <code>hr-docverify...</code>) in mail and proxy	☐ Block high-risk scopes for user consent (<code>Mail.*</code> , <code>Files.*.All</code> , <code>offline_access</code> combos)
☒ Disable the service principal; revoke refresh tokens and sessions for both users	☐ Review third-party apps with mail/file scopes for unusual API volume	☐ Conditional access for workload identities; token-protection / continuous access evaluation where licensed
☒ Review sign-in history, mailbox activity (340 items), file access (23 files) — hand the list to data owners	☐ Review privileged cloud accounts and admin-consented apps	☐ App-governance alerting: new app + data scopes, multi-user consent, app activity anomaly
☒ Block the domain and IP; purge the lure mail	☐ Check inbox rules, forwarding, delegates on affected mailboxes	☐ Application allow-listing for sensitive groups (finance, HR, executives)
☒ Preserve logs before retention expires (Entra audit, sign-in, unified audit, Graph activity)	☐ Correlate identity with endpoint telemetry for the two users' devices	☐ CTI-to-SOC enrichment: app IDs / redirect domains as first-class indicators
☒ Start targeted identity + endpoint hunting; warn treasury about payment-change requests	☐ Identify excessive app permissions tenant-wide	☐ Consent-phishing module in awareness training; quarterly app review

Order matters on day one: **revoke the grant and tokens first, then reset the password** — a password reset alone leaves the application's access intact.

Mission 09 — CISO Intelligence Brief (one page)

Executive summary. Two employees were tricked into approving a fake "document verification" app, which then read finance email and treasury and board files directly from our cloud — without stealing a password or defeating MFA. It matters because the access looks like a normal approved integration and survives a password reset until the approval itself is removed. Leadership should understand this is a *policy gap* (any employee can approve any app), not a user-carelessness problem, and it can be closed with one configuration decision.

Who is at risk? Finance users (payment and treasury data) · executives and their assistants (board material, delegated mailboxes) · administrators (tenant-wide consent) · any high-value cloud account · users with large or shared mailboxes · users with access to sensitive document libraries (HR, legal, M&A).

Business impact.

Impact	Assessment
Email compromise	Confirmed — 340 mail items read from one finance mailbox
Sensitive document exposure	Confirmed — 23 treasury / board-pack files accessed
Data theft	Likely — downloads recorded; total volume to be confirmed
Business Email Compromise	Elevated for the next weeks: attacker now knows vendors, invoice formats and approval chains
Cloud account takeover	Partial — delegated access to two identities; no admin role observed
Regulatory	Possible notification duty if personal/HR data was in scope (DPDP Act; GDPR for EU data subjects) — legal to assess
Reputation	Moderate, rising sharply if a fraudulent payment or leak follows

What should the SOC do? *Next 24 h:* contain — revoke grants and tokens, disable the app, scope both users, preserve logs, hunt the app ID and lure domain. *Next 7 days:* investigate — historical consent review, related infrastructure, inbox rules, other apps with similar scopes. *Next 30 days:* harden — restrict user consent, admin-consent workflow, app governance, conditional access for apps, quarterly review.

Confidence + source reliability. - ● **High — this is malicious consent.** Direct first-party audit evidence, five independent signals that agree (first-seen app, unverified publisher, foreign non-interactive access, out-of-baseline data, second victim). - ● **Medium — motivation is financial / BEC preparation.** Inferred from what was read first; no fraud attempt observed yet. - ● **Low — who is responsible.** New infrastructure, no overlap with reported clusters. We do not attribute.

 Senior Intel question — malware on a workstation (A) vs OAuth application (B)

Neither is "harder" in every dimension; they fail defenders in different places.

Dimension	A — obvious malware on a workstation	B — seemingly legitimate OAuth application
Detection window	Short: EDR/AV act at execution, usually minutes to hours	Long: nothing executes locally; found only if someone reviews consents or app behaviour
Visibility	High on the endpoint — process, file, network, memory	Only in identity/cloud audit logs, which many SOCs collect but do not hunt
Persistence	Needs a mechanism (service, task, registry) that can itself be detected and is removed by reimaging	The refresh token <i>is</i> the persistence; survives reimage and password reset
User trust	A warning or odd behaviour may alert the user	The user saw a genuine Microsoft screen and believes they opened a document
MFA limitations	MFA still protects the account from the attacker's own sign-ins	MFA is satisfied before the grant and irrelevant afterwards
API activity	Rare; traffic leaves from the infected host	All activity is API calls from attacker infrastructure, blending with hundreds of real integrations
False positives	Low for obvious malware — signatures and behaviour are specific	High — new SaaS, marketplace apps and automation tools look the same at first sight
Incident-response cost	Bounded: isolate, image, rebuild, reset	Wider and less certain: every mail item and file the scope allowed must be treated as exposed, across every consenting user

Argument. A is harder for the *attacker* to keep quiet — it produces evidence at the moment of execution, in the place defenders watch most. B is harder for the *defender* to notice — it produces evidence only in logs that need a baseline to interpret, and each event is individually legitimate. In this lab the EDR saw nothing because there was nothing to see; the intrusion was confirmed only by correlating consent → foreign API → out-of-baseline data → second user. But B is also the **easier one to prevent**: one consent policy removes the whole class, while no single setting removes malware. The honest conclusion: **B is harder to detect, A is harder to prevent — so invest in endpoint detection for A and in consent governance plus identity hunting for B.**

MITRE ATT&CK map

T1566.002 · T1204.001 · **T1528 Steal Application Access Token** · T1550.001 · T1078.004 · T1526 / T1538 · **T1114.002** · **T1213.002** · T1098 / T1098.003 (hunt) · T1567

Sources

Microsoft Learn — *Protect against consent phishing* (Entra ID) and *Detect and remediate illicit consent grants* (Defender for Office 365) · Microsoft Security Blog, 25 Jan 2024 — *Midnight Blizzard: guidance for responders on nation-state attack* (OAuth application abuse) · Proofpoint research, Jan 2023 — malicious OAuth apps abusing "verified publisher" status (as reported by SecurityWeek / The Register) · MITRE ATT&CK T1528, T1078, T1566, T1114, T1213 · CISA cloud-security guidance. Public reporting is cited as **background on the technique only — not as attribution** for this lab campaign. All lab data is synthetic; no private accounts, purchases or authentication attempts were involved.